Building a Multi-Layer Knowledge Graph for Waste Disposal Guidance



Study program

Wirtschaftsinformatik Master WiSe 2025/2026

Course

Data Engineering

Professor

Prof. Dr. Josef Fink

Submitted by

Riel, Niklas (1253801)
Hashemi, Arvand (1360176)
Rao, Niraj (1358243)
Cefariello, Luke (1350053)

Table of Contents

List of Abbreviations **3**
Executive Summary **5**
Introduction **5**
Initial Situation 6
The Case for Graph-Based Retrieval 6
Project Scope 7
Literature **8**
Related Work 8
Property-Graph Modeling and Schemas 8
Graph Construction Pipelines and Metadata-Rich Ingestion 8
Entity Resolution and Alignment for Multi-Source Knowledge Graphs 9
Storage Formats and Large-Scale Processing for Analytical Pipelines 9
Retrieval Pipeline Engineering for RAG Systems 10
Orchestration Patterns and Multimodal Ingestion Components 10
Cleaning Up the Implementation **11**
Usability Aspects of Agentic Chatbots 11
The Feedback Problem 11
Visible Reasoning: The Thought Process Panel 12
Streaming Architecture 12
Extending the Agent's Toolset 13
Switching to a Reasoning Model 13
Knowledge Graph Tools 14
Web Search 14
Vision Helper 14
EAN Barcode Scanner 15
The Impact of Unconstrained Reasoning 15
Designing the Graph Database Schema **16**
Design Principles 16
Layer separation 16
LLM-friendliness 17
Core versus professional layers 17
The Three-Layer Ontology 17
Core Entities: What Is It? 17
Logic and Rules: How Should It Be Disposed Of? 18
Infrastructure: Where Does It Go? 18
Schema Evolution: The Vocabulary Layer 18
Schema Evolution: The Product Decomposition Layer 19
The Final Schema 20
Creating the Dictionary Baseline **21**
The Case for Top-Down Construction 22

The Two-Label Strategy 23
Data Sources 23
The Import Pipeline 24
Phase 1: Download and Cache 24
Phase 2: Parse and Filter 24
Phase 3: Neo4j Import 25
Phase 4: LLM Classification and Promotion 25
Results 25
Integration with Existing ETL 27
Infrastructure 27
Why Python 27
Neo4j Deployment 28
JupyterLab 28
Database Management 29
The Chatbot Deployment 29
Reproducing the Setup 30
Results 30
Data Engineering Perspective 31
Schema as a living design 31
The dictionary baseline as foundation 31
Product decomposition chain 32
Infrastructure 32
Chatbot improvements 32
Future Work 32
Immediate Next Steps 33
Completing the AVV classification 33
Municipal vocabulary expansion 33
Data Source Integration 33
Municipal SaaS APIs 33
Chemical substance databases 34
LLM extraction from unstructured documents 34
Institutional Partnerships 34
University procurement catalogs 34
University-specific infrastructure data 35
Connecting Products to Disposal Knowledge 35
Graph Quality and Maintenance 35
Material taxonomy extension 35
Classification refinement 36
References 36

List of Abbreviations

Abbreviation	Meaning
API	**A**pplication **P**rogramming **I**nterface
ASR	**A**utomatic **S**peech **R**ecognition
CC BY-SA	**C**reative **C**ommons - **BY** (Attribution) - **SA** (ShareAlike)
CPU	**C**entral **P**rocessing **U**nit
CSV	**C**omma-**S**eparated **V**alues
EAN	**E**uropean **A**rticle **N**umber
ETL	**E**xtract, **T**ransform, **L**oad
GFDL	**G**NU **F**ree **D**ocumentation **L**icense
GPT	**G**enerative **P**re-**T**rained **T**ransformer
GPU	**G**raphics **P**rocessing **U**nit
GraphRAG	**Graph** + **R**etrieval-**A**ugmented **G**eneration
GTIN	**G**lobal **T**rade **I**tem **N**umber
HTTP	**H**yper**t**ext **T**ransfer **P**rotocol
HTTPS	**H**yper**t**ext **T**ransfer **P**rotocol **S**ecure
JSON	**J**ava**S**cript **O**bject **N**otation

JSONL	**JSON L**ines
LLM	**L**arge **L**anguage **M**odel
OCR	**O**ptical **C**haracter **R**ecognition
PWA	**P**rogressive **W**eb **A**pp
RAG	**R**etrieval-**A**ugmented **G**eneration
SQL	**S**tructured **Q**uery **L**anguage
SSE	**S**erver-**S**ent **E**vents
UID	**U**nique **Id**entifier
UPC	**U**niversal **P**roduct **C**ode
URL	**U**niform **R**esource **L**ocator
W3C	**W**orld **W**ide **W**eb **C**onsortium
XML	e**X**tensible **M**arkup **L**anguage

Executive Summary

This report documents the data engineering work conducted for the Fessi waste disposal chatbot. The project inherited a system with only 548 waste items sourced from a single CSV file and a vector-based retrieval pipeline that could not handle the conditional logic inherent in waste disposal queries, where the same item may require fundamentally different handling depending on its state or material composition. Our semester's goal was to build the knowledge graph infrastructure that enables graph-based retrieval, replacing flat document lookups with structured traversal of decision trees, disposal rules, and physical collection points. We began by redesigning the chatbot agent itself, replacing the rigid fixed-step pipeline with a freely reasoning model, implementing a visible reasoning panel with SSE streaming so users could observe the agent's multi-step process, and adding knowledge graph tools, web search, vision capabilities, and an EAN barcode scanner that decomposes real products into per-component disposal instructions. We then designed a five-layer graph schema that separates vocabulary, core waste entities, product decomposition, conditional disposal logic, and physical infrastructure into distinct layers, each addressing a specific class of query. The schema evolved iteratively from three layers to five as real data exposed modeling gaps, yet the core traversal pattern survived every extension unchanged. The most consequential architectural decision was building a dictionary baseline before importing any domain-specific data. Using structured exports from the German Wiktionary, the Open German WordNet, and the Open English WordNet, we constructed 329,231 concept clusters containing synonyms, translations, and taxonomic relationships, then classified each cluster with a small language model to determine whether it represents a physical, disposable object. This promoted 67,912 entries to WasteItem status, a 124-fold increase over the inherited vocabulary, and established a broad anchor layer that prevents duplicate explosion in subsequent imports. The semester concluded with a knowledge graph of approximately 5.3 million nodes and 6 million edges, populated by nine import pipelines, served from a single workstation running rootless Podman containers, and immediately available to the chatbot agent. All pipelines are idempotent, provenance-tracked, and reproducible from open-access data sources at negligible cost. The remaining work is primarily data integration: completing the AVV classification across all 67,000 WasteItem nodes, importing municipal disposal rules via three identified SaaS platform APIs, processing chemical substance databases for laboratory waste, and connecting the 4.29 million imported product nodes to disposal knowledge through category-based and LLM-based matching.

Introduction

The effective management of waste disposal on university campuses presents a deceptively complex challenge. What appears straightforward ("Where do I throw this away?") quickly reveals layers of conditional logic, regional regulations, and situational context that no simple lookup table can address. A paint can, for instance, may require fundamentally different disposal routes depending on whether it still contains liquid paint or is empty and dry, a distinction that determines whether it constitutes hazardous waste or ordinary recyclable packaging. This kind

of conditional reasoning, common across hundreds of waste categories, is precisely where traditional information retrieval systems fail.

"Fessi" is an intelligent waste disposal chatbot developed for the Frankfurt University of Applied Sciences campus. The project, now in its third semester of iterative development, aims to provide students, staff, and visitors with precise, context-aware disposal guidance through a conversational interface. The first semester produced "Müll-Meister," an initial prototype built on jQuery and Microsoft frameworks that demonstrated the concept but suffered from fundamental architectural limitations, including initialization times exceeding 58 seconds and a monolithic backend structure. The second semester saw a complete rewrite: our team rebuilt the frontend as a Progressive Web App using Angular, developed the API specifications, and delivered a fully functional chat interface with voice input, image upload, and offline capabilities. Concurrently, the backend was migrated to a LangGraph-based agent architecture, but the underlying retrieval mechanism remained a rigid, vector-based RAG pipeline that was not designed to handle the conditional logic inherent in waste disposal queries. As a result, the chatbot's performance was severely hampered by long response times and high token usage due to the need to retrieve and process large amounts of often irrelevant information, rendering it effectively unusable.

Initial Situation

Upon beginning this semester's work, we inherited a system that was architecturally sound but operationally limited. The previous backend/pipeline team had migrated the chatbot from a fixed processing pipeline to a LangGraph-based agent (a significant step forward). However, the agent still operated under strict guardrails inherited from the original pipeline design: a rigid sequence of vector search, follow-up question evaluation, followed by yet another vector search before starting the final response formulation. Since the scope and in particular the long inference times had not been shared with the other groups, they were not included into the design specifications, resulting in response times of up to 30 seconds during which the user interface provided no feedback whatsoever. Something which hurt the usability to a point that it rendered the chatbot effectively "unusable". More critically, upon examining the underlying knowledge base, we found remarkably little to build upon. After sorting/importing everything available, the system contained approximately 548 waste items sourced almost entirely from the Abfall-ABC. This limited coverage meant that the majority of user queries simply could not be answered from the knowledge base. In practice, the agent would search the graph or vector store, find nothing relevant, and answer from its parametric knowledge which often led to generic responses who weren't grounded in the actual data.

The Case for Graph-Based Retrieval

The existing system relied on vector-based Retrieval-Augmented Generation (RAG), which retrieves document chunks by semantic similarity and passes them to the language model for response generation. This approach works well for straightforward factual lookups ("What are the opening hours of Wertstoffhof Nord?") where the answer exists verbatim in a document.

However, it fundamentally fails on conditional disposal logic, which accounts for a significant portion of real waste disposal queries.

Consider the "Paint Can" problem: a user asks "How do I dispose of a paint can?" A vector search retrieves all document chunks mentioning paint cans, which simultaneously contain conflicting information: "dried paint residue goes in residual waste," "liquid paint must be taken to a hazardous waste facility," "empty metal cans belong in the packaging bin." The language model receives these contradictory passages and must somehow synthesize a coherent answer, typically resulting in a vague response that lists all options without determining which applies.

A knowledge graph models this problem as a decision tree rather than a collection of documents. The waste item "paint can" connects to a disposal rule, which branches on a condition ("Is it still liquid?"). The true branch routes to the hazardous waste facility; the false branch routes to the packaging bin. When the chatbot agent traverses this graph, it encounters the condition node and can ask the user a clarifying question before providing a precise, unambiguous answer. This is not a theoretical advantage; it represents the difference between "here are some things you could do with a paint can" and "bring it to the Schadstoffannahme on Campus Nibelungenplatz, open Tuesday and Thursday 10–14 Uhr."

Beyond conditional logic, graph-based retrieval offers structural advantages for an agentic chatbot. Rather than retrieving a fixed number of similar text chunks, the agent can traverse relationships, following links from a waste item to its disposal rules, from rules to facilities, from facilities to opening hours and addresses. This traversal mirrors the way a knowledgeable human would reason about waste disposal: identify the item, determine its category, check for special conditions, and locate the appropriate disposal point. A knowledge graph makes this reasoning path explicit and queryable.

Project Scope

This semester's work focuses on building the knowledge graph infrastructure that the chatbot needs to provide accurate, context-aware disposal guidance. This encompasses two complementary workstreams: constructing ETL pipelines that transform raw data sources into a structured Neo4j knowledge graph, and developing LLM-based extraction methods that can process unstructured German documents into graph-compatible entities and relationships.

The project also addresses the data scarcity problem identified in the initial situation. Through systematic research, we identified roughly 400 viable data sources for waste disposal knowledge, ranging from chemical substance databases with millions of entries to municipal PDF guides and legal statutes. Importing and classifying this data at scale, using both deterministic pipelines for structured sources and LLM-based extraction for unstructured documents, forms the core technical contribution of this work.

In parallel, we made targeted improvements to the chatbot agent itself. We replaced the rigid pipeline with a freely reasoning agent, removing the guardrails that had previously constrained it to a fixed sequence of tool calls. We implemented visible reasoning steps (inspired by the "thinking" displays in modern AI interfaces) so that the user could follow the agent's progress rather than staring at a blank screen. We added web search as a fallback knowledge source, and built a barcode scanning module that can decompose real-world products into their individual packaging components for per-material disposal routing, built directly into the knowledge graph.

This documentation is split across two companion documents reflecting the two modules under which this work was conducted. The present document covers the **Data Engineering** aspects: data sources and quality assessment, graph schema design, structured ETL pipelines, the dictionary baseline data pipeline, the EAN barcode product pipeline, and infrastructure.

Literature

Related Work

Property-Graph Modeling and Schemas

Property graphs represent domain knowledge as nodes and directed edges, where both nodes and edges can carry properties (key–value attributes) and labels/types that support typed modeling and later traversal. Recent schema work for property graphs formalizes how such graphs can be constrained and validated by defining node/edge types, property domains and ranges, and integrity constraints, so that graph instances can be checked for conformance rather than relying on “schema-by-convention” alone (Angles et al., 2023).

In practitioner-oriented knowledge-graph engineering, a recurring recommendation is to begin with a small set of core entity and relation types and evolve the schema iteratively as extraction and linking results expose missing concepts or inconsistencies. This schema-first, iterative refinement is motivated by the fact that early modeling decisions influence later retrieval behavior and the interpretability of answers produced by graph-backed assistants (Curry, 2026).

Graph Construction Pipelines and Metadata-Rich Ingestion

Knowledge graph construction from heterogeneous sources is commonly framed as a staged pipeline that transforms raw documents into structured graph objects via extraction, normalization, and linking, followed by validation and curation loops. Rather than assuming a single-pass build, practitioner guidance emphasizes iterative correction cycles to improve entity consistency and reduce accumulated modeling errors as the corpus grows (Curry, 2026).

For retrieval-centered applications, Raju emphasizes that metadata is not optional: fields such as document source, date, author, topic, and version allow the retrieval system to filter and rank results intelligently before semantic search even begins. This shifts the engineering focus from "just storing text" to maintaining structured records so that the model does not need to compensate for missing structure. (Raju, 2026)

Repeatability is also treated as an engineering property of the pipeline: designs that persist intermediate artifacts (e.g., parsed documents, chunk boundaries, structural pointers) reduce recomputation cost and support systematic error analysis by allowing each stage's outputs to be inspected and reproduced across runs (Silfverskiöld, 2025).

Entity Resolution and Alignment for Multi-Source Knowledge Graphs

When integrating multiple knowledge graphs, entity alignment (also referred to as entity matching/resolution in parts of the literature) targets the identification of entities across graphs that correspond to the same real-world object despite heterogeneity in identifiers, language, and schema. Survey work synthesizes a broad spectrum of approaches, spanning traditional similarity-based and rule-based techniques as well as representation-learning and graph-neural-network-based methods that embed entities into vector spaces to enable scalable matching under heterogeneity (Zeng et al., 2021).

Incremental approaches address the practical requirement that real pipelines continuously ingest new sources. Work on incremental multi-source entity resolution for knowledge graph completion proposes update-oriented mechanisms that avoid full recomputation by maintaining and repairing clusters as additional evidence arrives, aligning better with continuous ingestion settings than strictly batch-oriented resolution (Saeedi et al., 2020).

Storage Formats and Large-Scale Processing for Analytical Pipelines

At scale, the choice of data format has direct implications for I/O efficiency, compression behavior, and query execution strategies. A recent survey on analytical DBMS formats discusses performance trade-offs between row- and column-oriented layouts and highlights how encoding, compression, late materialization, and vectorized execution interact with storage formats to shape end-to-end analytical performance (Liu et al., 2025).

For graph data in data lakes, dedicated storage layouts are proposed to bridge graph access patterns with columnar lakehouse infrastructure. GraphAr introduces a storage scheme for graph data in data lakes that targets interoperability and analytical workloads by mapping graph structures onto data-lake-friendly representations, enabling graph processing while retaining benefits of columnar storage systems (Li et al., 2024).

Retrieval Pipeline Engineering for RAG Systems

In production-oriented RAG discussions, Raju argues that retrieval should be treated as a ranking problem rather than a simple lookup. He describes a multi-stage process, broad candidate retrieval, metadata filtering, and re-ranking, followed by deliberate context assembly (e.g., logical ordering, deduplication, and lightweight structure) so that fewer but more meaningful chunks reach the model and the context becomes easier to use reliably. (Raju, 2026)

More elaborate retrieval-system designs extend this pipeline with hybrid retrieval, query rewriting or expansion, and context reconstruction strategies to recover coherence in long documents. These additions are presented as targeted responses to observable failure modes, but they also increase latency and maintenance complexity, which motivates careful justification of each stage (Silfverskiöld, 2025).

Recent research argues that limitations of chunk-based vector retrieval become especially visible for “global” questions that require corpus-level synthesis rather than local fact lookup. GraphRAG addresses this by building an entity-centric graph and community summaries during indexing and answering queries via aggregation over these graph-derived summaries, improving coverage and diversity of retrieved evidence for such sensemaking tasks.The approach also highlights chunking as a non-trivial design trade-off because context fragmentation and “lost-in-the-middle” effects can directly impact retrieval quality (Edge et al., 2024).

Rerankers are frequently highlighted as a pragmatic quality lever: instead of increasing top-k aggressively (which increases latency and noise), a bounded candidate set is retrieved and then reranked with a stronger model to improve the precision of the final context window (Singh, 2026).

Orchestration Patterns and Multimodal Ingestion Components

Agentic workflow pattern catalogs describe how assistant applications combine retrieval with control logic such as planning, tool use, reflection, and iterative reason act observe loops. These sources emphasize that orchestration patterns can improve robustness when applied selectively, but also warn that increasing agentic complexity makes debugging and evaluation harder (Deep concept, 2026). Beyond practitioner pattern catalogs, survey research on tool learning structures LLM based orchestration into four recurring stages: task planning, tool selection, task execution, and response generation. This framework provides a consistent lens to compare orchestration designs and typical failure modes. The survey also notes that increasing tool and control complexity can improve robustness but often raises latency and makes debugging and evaluation harder, which motivates selective use of orchestration steps (Chen et al., 2025).

For multimodal ingestion, automatic speech recognition can act as a front end normalizer that converts audio into text for downstream processing. Whisper demonstrates robust speech recognition via large scale weak supervision, supporting the practical pattern of transcribing audio inputs into text so they can be handled by the same extraction, indexing, and retrieval pipeline used for textual sources (Radford et al., 2022).

Cleaning Up the Implementation

Given the initial situation (an agent constrained by inherited guardrails, response times of up to 30 seconds with no user feedback, and a knowledge base too sparse to answer most queries), we concluded that incremental improvements to the existing system would not be sufficient. Instead, we decided to rebuild the chatbot's core interaction model from scratch. The guiding insight was that the very behavior which had rendered the chatbot "unusable," its tendency to make multiple tool calls and reason through a problem step by step, was not a defect to be suppressed, but a capability to be surfaced. The previous implementation had hidden this process behind a blank screen, our task was to make it visible.

Usability Aspects of Agentic Chatbots

Before addressing the technical implementation, we examine the usability principles that guided our redesign. Specifically, how agentic chatbots can maintain user trust during multi-step reasoning.

The Feedback Problem

The original chatbot suffered from what is, in retrospect, a straightforward usability failure rather than a performance problem. As described in the previous section, the agent's fixed pipeline of vector searches and evaluations could take 20 to 30 seconds per query. During this entire period, the user interface displayed nothing: no progress indicator, no status message, no evidence that the system was working at all. From the user's perspective, the chatbot appeared to have frozen.

Our initial assumption was that reducing response time would solve this. After working with the system, however, we came to a different conclusion: the wait itself was not the core issue. Users routinely wait 10 to 30 seconds for complex queries on modern AI interfaces. ChatGPT, Gemini, and Perplexity all exhibit similar latencies for multi-step reasoning tasks. What those systems provide, and what our chatbot lacked, was visibility into the process. A user who can see that

the agent is "Searching the knowledge base for Batterien" and then "Exploring waste categories" perceives an active system working on their behalf. A user staring at a blank screen perceives a broken one.

Visible Reasoning: The Thought Process Panel

Inspired by how Google's Gemini displays an expandable summary of its internal reasoning steps, we implemented a "Thought process" panel that renders the agent's actions in real time as collapsible accordion elements. Each action the agent takes (thinking, calling a tool, receiving a result) appears as a distinct step in a vertical timeline, color-coded by type.

- **Thought steps:** represent the agent's internal reasoning: "Analyzing request..." These appear whenever the language model is invoked to decide what to do next.
- **Tool calls:** show which tool the agent is invoking and with what parameters. For example, a knowledge base search displays the JSON input {"waste_item": "Batterien"}, making the agent's search strategy transparent.
- **Tool results:** display the outcome of each tool call, including the execution duration. Users can expand these to inspect the raw data the agent received.
- Observations: appear when the agent synthesizes intermediate findings before deciding on its next action.

The panel header shows a summary, "Thought process (20 steps)," along with a status indicator: a spinner while the agent is still working, a checkmark once it has finished. The entire panel is collapsible: users who want the quick answer can ignore it; users who want to understand how the agent arrived at that answer can expand individual steps and trace the full reasoning chain.

This design serves two purposes simultaneously. For the end user, it transforms an opaque wait into an observable process. For developers and domain experts, it provides a debugging and evaluation interface: one can see exactly which tools the agent called, what data it received, and where it may have gone wrong.

Streaming Architecture

To deliver reasoning steps and response text incrementally rather than as a single payload after completion, we replaced the chatbot's request-response communication model with Server-Sent Events (SSE). The backend streams four event types to the frontend:

1. **Step** events carry individual agent actions (thought, tool call, tool result) as they occur. Each step includes a type, title, content, timestamp, and, for tool results, execution duration and metadata.
2. **Token** events carry individual text tokens of the final response, enabling the familiar character-by-character typing effect.
3. **Message_start** provides metadata (message ID, conversation ID, timestamp) at the beginning of the stream.

4. **Done** signals completion so the frontend can finalize the message.

On the backend, the FastAPI endpoint hooks into LangGraph's event system. Each LangGraph callback (on_chat_model_start, on_tool_start, on_tool_end, on_chat_model_stream) maps to an SSE event that is pushed to the client immediately. On the frontend, an Angular streaming service consumes these events using the native fetch API with an AbortController for cancellation support, parsing the SSE protocol and routing each event type to the appropriate UI component.

The result is that the first reasoning step appears within one to two seconds of the user submitting a query. Even when the full response takes 30 seconds or more, the user sees continuous activity from the moment they press send.

Extending the Agent's Toolset

With the usability foundation in place (users could now observe and tolerate multi-step reasoning), we turned to expanding what the agent could actually do. The inherited agent ran on GPT-4o and had access to a single tool: vector similarity search over the knowledge base. We changed the underlying model and added four new tools.

Switching to a Reasoning Model

We replaced GPT-4o with a reasoning model (GPT-OSS-120B). The motivation was not raw capability but behavioral: reasoning models engage in explicit chain-of-thought before producing an output, which fundamentally changes how they interact with tools. Where GPT-4o required careful prompt engineering to follow a multi-step tool-use strategy, the reasoning model autonomously plans its approach, deciding which tools to call, in what order, and how to synthesize the results, with minimal instruction.

This had a direct consequence for the guardrail problem. The previous system needed a rigid pipeline (search, evaluate, search again, respond) precisely because the non-reasoning model could not reliably decide for itself what to do next. With a reasoning model, we could remove these guardrails entirely and let the agent determine its own strategy for each query. In practice, we observed the agent performing up to 30 tool calls on complex queries, searching the knowledge base, refining its search terms based on initial results, cross-referencing facilities, and only then formulating a response. This behavior would have been impossible under the old fixed-step pipeline and prohibitively slow without the streaming architecture described above.

The system supports configurable reasoning levels (low, medium, high) via environment variables, allowing the deployment to balance thoroughness against latency depending on the use case.

Knowledge Graph Tools

We added tools that allow the agent to query and navigate the Neo4j knowledge graph directly. Rather than relying solely on vector similarity search, which returns text chunks ranked by embedding distance, these tools let the agent execute structured queries: looking up a waste item by name, retrieving its disposal rules and associated conditions, exploring connected facilities with their addresses and opening hours, and traversing the category hierarchy. This is the mechanism through which Graph-RAG delivers its advantage over vector-RAG: the agent does not receive a list of similar documents, but traverses a structured network of relationships.

Web Search

We integrated web search via the Tavily API as a fallback knowledge source. When the knowledge base does not contain the information the agent needs, which, given the data scarcity, was frequently the case, the agent can issue targeted web queries and incorporate the results into its response.

This was initially intended as a stopgap to compensate for the sparse knowledge base. In practice, it proved more valuable than expected: it allowed us to test and refine the agent's reasoning capabilities without being blocked by missing data. We could observe how the agent narrowed its search queries progressively, starting broad, then targeting specific details it had identified as missing, which closely mirrors how it would navigate a well-populated knowledge graph. When asked about disposing of hydrochloric acid in Bremen, for instance, the agent issued three successive web searches: first for general chemical disposal regulations, then for the specific hazardous waste collection service, and finally for the address and opening hours of the nearest collection point. This iterative refinement pattern reinforced our confidence in the graph-based architecture, since it demonstrated that the agent could effectively navigate a structured information landscape to assemble a complete answer from multiple targeted queries.

Vision Helper

The reasoning model we selected (GPT-OSS-120B) is not multimodal, meaning it cannot process images directly. Since the chatbot's frontend supports image uploads (a capability built in the previous semester), we implemented a vision helper service that delegates image analysis to a separate multimodal model. When the agent receives a message containing an image, the vision helper sends it to a dedicated vision model (configurable, defaulting to GPT-4o-mini) which generates a text description. The agent then reasons about this description using its primary model. The tool also has a question parameter which allows the agent to issue targeted followup questions on an image.

This architecture decouples the choice of reasoning model from the requirement for multimodal input. It also enables deployment on heterogeneous hardware: in our development environment, we ran the vision model (Qwen2.5-VL 7B) on one GPU and the reasoning model on another, with audio transcription (via faster-whisper) on the CPU.

EAN Barcode Scanner

Building on the vision helper, we developed a barcode scanning module that allows users to photograph product barcodes for disposal guidance. The module uses pyzbar (a Python wrapper around the ZBar library) for barcode detection and decoding. A deterministic computer vision step (not an LLM call). The extracted EAN number is then passed to a multi-API resolver that queries product databases in a cascading fallback sequence: Open Food Facts, Open Beauty Facts, UPCitemdb, and optionally OpenGTINDB. The first API to return a match provides the product data, including, critically, a breakdown of packaging components with their materials.

This capability transforms a single user action (photographing a product) into per-component disposal routing. An empty jar of Nutella, for instance, decomposes into a glass jar (Altglascontainer), a polypropylene lid (Verpackungstonne), and a paper label (Altpapiertonne). The barcode pipeline and the underlying product-to-material-to-waste-stream graph traversal are described in detail in the EAN barcode pipeline section later in this document. The barcode pipeline and the underlying product-to-material-to-waste-stream graph traversal are described in detail in the EAN barcode pipeline section later in this document. The scanner is fully integrated into the live chatbot as an agent tool. The user can trigger it from the attachment menu, the detected EAN is passed to the lookup_product_barcode tool, which queries the knowledge graph first and falls back to external APIs, so that the agent can continue with per-component disposal instructions.

The Impact of Unconstrained Reasoning

The combination of visible reasoning, streaming, and an expanded toolset produced a qualitative shift in the chatbot's behavior. Under the old system, the agent followed a script: search, evaluate, search, respond. Under the new system, the agent adapts to the query. A simple factual question ("What goes in the Biotonne?") might require only a single knowledge base lookup. A complex conditional question ("How do I dispose of a half-full paint can?") triggers a multi-step investigation: searching for the item, identifying the relevant condition, checking both branches of the decision tree, and locating the appropriate facility. A question that falls outside the knowledge base's coverage might prompt the agent to fall back to web search, querying multiple sources and synthesizing the results into a structured answer.

This flexibility came not from more complex prompt engineering, but from less. The reasoning model's internal chain-of-thought replaced the external guardrails that had previously been necessary to guide a non-reasoning model through multi-step tasks. With fewer constraints, the agent could recover gracefully from dead ends (for instance, falling back to web search when a graph query returned no results) rather than failing silently at a predetermined step in a fixed pipeline.

The visible reasoning panel makes this adaptive behavior transparent to the user. Where the old chatbot was a black box that either returned an answer or not, the new system exposes its decision-making process. A user asking about battery disposal can watch the agent search for

“Batterien”, discover that batteries are classified as hazardous waste, look up the campus hazardous waste collection point, retrieve its address and opening hours, and compose a response that synthesizes all of this into actionable guidance. The answer takes longer to produce, but the user understands why, and trusts the result more because they witnessed the reasoning that went into it.

Designing the Graph Database Schema

The chatbot's knowledge graph must answer a deceptively layered question: "How do I dispose of X?" The answer is rarely a single fact. It depends on what the item is made of, what state it is in, which regulations apply, and where the nearest appropriate collection point is located. A banana peel goes in the Biotonne: one lookup, one answer. A paint can require knowing whether it still contains liquid paint (hazardous waste collection) or is empty and dry (packaging bin), and in either case the user needs the address and opening hours of the relevant facility. An electric toothbrush on campus might follow different rules than the same item at home, because the university can operate its own e-waste collection point that should override the city-level default. These three cases impose distinct requirements on the schema.

- The banana peel requires only a direct lookup: item -> stream
- The paint can requires conditional branching: item -> rule -> condition -> stream
- The toothbrush requires scope-aware override logic: item -> campus rule (which supersedes city rule) -> stream -> campus collection point

A flat data model, a table of items and their disposal routes, cannot express any of these patterns beyond the first. The schema we designed had to encode all three, and it had to do so in a way that a language model could traverse through natural-language tool calls. This section describes the resulting ontology and the two major extensions it underwent as our data strategy evolved.

Design Principles

We derived the schema from various aspects such as the problem space, the entity types or relationships we anticipated, then iteratively refined the structure over the course of the semester. The following three principles helped to guide the design.

Layer separation

The schema separates what something is (core entities), how it should be disposed of (rules and conditions), and where disposal physically happens (infrastructure). This separation mirrors how a human reasons about waste disposal: first identify the item, then determine the applicable rule, then locate the collection point. It also means that changes to campus infrastructure (a new bin location, a relocated collection point) do not require modifying any disposal rules, and vice versa.

LLM-friendliness

Since the chatbot agent traverses the graph through natural-language tool calls, the schema had to be legible to a language model. This meant few, semantically unambiguous relationship types (HAS_RULE, REQUIRES_CHECK, LEADS_TO, LOCATED_AT) rather than many specialized ones, and short, descriptive node labels that a model can reference in its reasoning. The schema was designed so that a system prompt listing the node types and their key relationships would be sufficient for the agent to formulate useful Cypher queries, with no specialized query templates needed.

Core versus professional layers

Not every node type is needed for a basic waste disposal answer. The minimum viable graph requires only WasteItem, WasteStream, DisposalRule, and Facility. More advanced features (material composition, AVV legal codes, hazard classifications, per-room container inventories) were designated as "professional layer" extensions that could be added incrementally without disrupting the core traversal pattern.

The Three-Layer Ontology

The initial schema organized the graph into three layers, each answering a distinct class of question.

Core Entities: What Is It?

The **WasteItem** is the central node in the graph and the entry point for every query. It represents a waste category as a user would name it: "Bananenschale," "Farbdose," "elektrische Zahnbürste". Each WasteItem carries a canonical German name (unique-constrained), an array of synonyms for fuzzy matching, and an optional description. The chatbot agent's first action on any query is to resolve the user's input to a WasteItem node.

WasteStream represents the logical disposal pathway, a bin type or waste fraction. Frankfurt operates five standard streams: Restabfalltonne (residual waste), Biotonne (organic), Verpackungstonne (packaging), Altpapiertonne (paper), and Altglascontainer (glass). Each stream carries metadata such as lid color, body color, and emptying frequency. Streams also distinguish between collection modes: "hol" (curbside pickup), "bring" (drop-off), and "service" (scheduled collection).

AVVCode encodes the legal classification under the Abfallverzeichnis-Verordnung (European Waste Catalogue). The AVV is a three-level hierarchy of 20 chapters, approximately 113 subgroups, and 1,505 specific waste types, where codes marked with an asterisk denote hazardous waste. Connecting WasteItems to AVV codes via CLASSIFIED_AS edges anchors the graph in the regulatory framework, a distinction that matters when the chatbot must explain why an item requires special handling, not just where to bring it.

Logic and Rules: How Should It Be Disposed Of?

This is the layer that solves the "Paint Can" problem described earlier. Rather than storing conflicting disposal instructions as flat text, the schema models conditional logic as a traversable decision tree.

A **DisposalRule** is the entry point for disposal logic. Each WasteItem can have multiple rules, one for the city-level default, another for campus-specific overrides, connected via HAS_RULE edges. Rules carry a scope property ("city," "campus," "generic") and a numeric priority, enabling the agent to determine which rule applies in a given context. A campus-scoped rule can explicitly override its city-level counterpart via an OVERRIDES relationship.

For items with straightforward disposal, the rule connects directly to a WasteStream via LEADS_TO with no conditions required. For conditional items, the rule connects to a Condition node via REQUIRES_CHECK. The Condition carries a human-readable question ("Ist die Farbe noch flüssig?") and a machine-readable key ("aggregatzustand"). The branching is binary: IF_TRUE and IF_FALSE edges route to either a WasteStream (terminating the traversal) or another DisposalRule (which may carry its own Condition, enabling multi-level decision trees). The paint can, for instance, first branches on whether the contents are paint or lacquer, then on whether the substance is liquid or dry, two chained conditions, each resolved by a single yes-or-no clarification from the user.

Instructions attach imperative handling steps to rules: "Verpackung ausspülen," "Deckel abtrennen," "Nicht in den Ausguss kippen." These do not branch; they are practical guidance that the chatbot should communicate alongside the disposal route. Tips serve a similar role at a broader scope, attaching sorting hints, exceptions, or best practices to WasteStreams, WasteItems, or Containers (e.g., "Blue glass goes in the green container").

Infrastructure: Where Does It Go?

The infrastructure layer maps the abstract disposal routes to physical locations. Facility nodes represent external collection points (Wertstoffhöfe, Schadstoffmobile, glass container sites) with addresses, opening hours, and contact information. On campus, the hierarchy is finer-grained: Building -> Room -> Container, where each Container is typed by its WasteStream (e.g., "Biotonne_240L") and placed at a specific Room.

The key relationship is AVAILABLE_AT, which connects a WasteStream to the Facilities where it can be deposited. Combined with the disposal rules, this creates the full traversal path the agent follows: identify the item, determine the applicable rule, evaluate any conditions, arrive at a WasteStream, and find the nearest Facility or Container where that stream is collected.

Schema Evolution: The Vocabulary Layer

The three-layer ontology was designed to model disposal knowledge. However it did not address the discovery problem: how does the chatbot agent find the right WasteItem node when

a user asks about "Tafelschokolade" but the graph only contains "Schokolade"? The original 548 items from the Abfall-ABC covered only a fraction of the terms users might employ.

This gap became acute when we developed the dictionary baseline strategy described later in this document. The idea was to seed the graph with structured dictionary data, tens of thousands of German nouns with synonyms, translations, and taxonomic relationships, as anchor nodes for the LLM extraction pipeline. But these dictionary entries are not all waste items: "Verwandtschaft" and "Demokratie" are valid German nouns that should never receive disposal routes.

We solved this with a two-label strategy. Dictionary entries are imported as DictionaryEntry nodes, a new Layer 0 (Vocabulary) beneath the original three layers. After LLM classification determines whether an entry represents a physical, disposable object, qualifying entries receive an additional :WasteItem label. This gives the agent four distinct response states:

1. A node with both :WasteItem and disposal edges: full disposal guidance
2. A node with :WasteItem but no disposal edges: "I know this is waste but don't have routing yet"
3. A node with only :DictionaryEntry: "This isn't something you'd dispose of"
4. No match at all: "I don't recognize this term"

The vocabulary layer also introduced SYNONYM_OF edges (same-language synonyms), TRANSLATION_OF edges (cross-language links between German and English entries), and IS_A edges (hypernym - hyponym hierarchies from WordNet). These relationships dramatically expand the agent's recall without polluting the curated WasteItem namespace.

Schema Evolution: The Product Decomposition Layer

The second major extension arose when we began working with real product data from Open Food Facts. The original schema assumed a one-to-one mapping between things people throw away and waste categories: a chocolate bar is "Schokolade," which goes in the Biotonne. But this model breaks down for packaged products. A jar of Nutella is not one waste item but at least three: a glass jar (Altglascontainer), a polypropylene lid (Verpackungstonne), and a cardboard label (Altpapiertonne), each routed to a different bin. Under the original schema, the chatbot agent would have had to infer this decomposition from its parametric knowledge, with no structured data to support it.

The Open Food Facts API confirmed that this decomposition data was readily available. Its packagings array returns per-component breakdowns for products: material, shape, weight, and recycling hints for each physical part of the packaging. For Nutella (EAN 3017620422003), the API returns four distinct components: a clear glass pot (199.94 g), a polypropylene lid (6.8 g), a non-corrugated cardboard backing (~2.2 g), and a paper-aluminum seal (~0.4 g). Each component carries a material tag that maps deterministically to a waste stream. We therefore

introduced three new node types as Layer 2 (Products), sitting between the core entities and the rules layer.

- **Product** represents a real-world commercial product identified by its EAN barcode, carrying properties like name, brand, quantity, and category tags.
- **PackagingComponent** represents a physical part of the packaging (the jar, the lid, the label) with shape, weight, and recycling hint properties.
- **Material** represents a standardized material type from a taxonomy of approximately 90 entries derived from Open Food Facts tags (e.g., en:clear-glass, en:pp-5-polypropylene). Materials are organized in an IS_A hierarchy (polypropylene IS_A plastic) that enables fallback routing: if a specific sub-material has no explicit stream mapping, the agent traverses up to the parent material.

The decomposition chain Product -> HAS_COMPONENT -> PackagingComponent -> MADE_OF -> Material -> DEFAULT_STREAM -> WasteStream allows the agent to answer "How do I dispose of Nutella?" with per-component precision, without any LLM inference about packaging materials. The product itself also links to the core layer via Product -> IS_A -> WasteItem, connecting the product's contents (chocolate spread -> Biotonne) to the disposal graph independently of its packaging.

This pattern drew on established ontology precedents: the CEON (Circular Economy Ontology Network) defines a similar Product -> ProductComponent -> Material hierarchy, and the Neo4j Bill of Materials pattern uses HAS_PART relationships with quantity properties for recursive composition trees. The novelty in our application is connecting the decomposition chain through to waste stream routing; no existing system we found combines barcode lookup, knowledge graph traversal, and per-component disposal guidance in a single pipeline.

The Final Schema

The schema evolved from three layers to five over the course of the semester, each extension motivated by a concrete problem encountered during implementation.

Layer	Name	Key Node Types	Purpose
0	Vocabulary	DictionaryEntry	Multilingual discovery, synonym matching
1	Core Entities	WasteItem, WasteStream, AVVCode	Central disposal concepts and legal classification
2	Products	Product, PackagingComponent, Material	Per-component disposal routing for packaged products
3	Logic &	DisposalRule,	Conditional disposal logic and handling

	Rules	Condition, Instruction, Tip	instructions
4	Infrastructure	Container, Room, Building, Facility, Source	Physical locations and provenance tracking

The complete schema, implemented in src/db/schema.cql, defines uniqueness constraints and indexes for all node types and documents the full set of relationship patterns. Provenance tracking via DERIVED_FROM edges to Source nodes applies across all layers, so every node imported by an ETL pipeline or extracted by an LLM records its origin, enabling both auditability and incremental re-extraction when source data changes.

The schema's most important property is that it supports a consistent reasoning pattern regardless of entry point. A query about any item, whether a raw waste category, a dictionary term, or a barcoded product, enters the graph at one of three points (WasteItem, DictionaryEntry, or Product) and follows the same traversal toward the answer: identify the item, resolve its disposal rule, evaluate conditions if any, arrive at a waste stream, and locate the nearest collection point. The five layers grew organically over the semester, but the traversal path they collectively support was present in the original three-layer design and survived every extension unchanged.

Creating the Dictionary Baseline

Before building individual import pipelines, we conducted a rough evaluation of the available data sources (documented in full in the companion Data Mining documentation). Using the schema layers as an organizing framework, we derived ~20 research topics across four priority tiers and investigated each systematically. The resulting catalog assessed ~550 sources on a rubric, judging them by relevance, data quality, format, and licensing. Key findings included Open Food Facts as the primary product data source (4.29 million products with per-component packaging breakdowns), the AVV code hierarchy for legal classification, chemical substance databases (PubChem, GESTIS) for lab waste, and the discovery that three SaaS platforms consolidate most German municipal waste management, reducing what appeared to be 400 custom integrations to three API endpoints. The evaluation also identified the most significant data gap: no open equivalent to Open Food Facts exists for everyday office and stationery items.

With over 400 sources spanning millions of potential entities, the natural next step would have been to start building ETL pipelines for the highest-priority sources (import Open Food Facts products, parse AVV codes from the legal text, extract disposal rules from municipal PDFs) and let the graph grow incrementally as each pipeline delivered its data.

However we did not take this approach. Before importing any domain-specific data, we needed to address a structural problem that would have compounded with every subsequent import: the

graph had no vocabulary foundation. The 548 Abfall-ABC items inherited from the previous semester covered only a tiny fraction of the terms people use when describing waste. Running the LLM extraction pipeline against hundreds of source documents without a broad anchor layer would mean that entity creation and entity resolution would happen simultaneously: the pipeline would need to both understand a document and decide whether each entity it encountered was new or already existed in the graph. These are fundamentally different tasks, and combining them would most likely degrade both.

The Case for Top-Down Construction

The conventional approach to building a knowledge graph from heterogeneous sources is bottom-up: process each document, extract entities and relationships, merge duplicates after the fact. This approach has five structural problems that become acute at scale.

First, **duplicate explosion**. Source A mentions "Schokolade," source B "Schokoriegel," source C "chocolate bar." Without pre-existing anchor nodes, the pipeline creates three separate nodes for what is semantically one item. Second, **inconsistent granularity**. One source discusses "Batterie," another "Knopfzelle," a third "AA-Batterie". Without a taxonomy, the pipeline cannot determine whether these are synonyms, siblings, or a parent-child hierarchy. Third, **deduplication is destructive**. Merging nodes after the fact requires fuzzy matching and risks losing relationship edges and breaking provenance links. Fourth, would be **limited recall**. A student searches "Tafelschokolade" but the graph only contains "Schokolade" because that term appeared in the source PDFs. Without upfront synonym seeding, the graph's recall is bounded by the vocabulary of whichever documents have been processed. Fifth, the **LLM carries two competing tasks**: understanding document content and resolving entities against the existing graph.

A 2025 study comparing vector-based RAG, GraphRAG, and retrieval over ontology-guided knowledge graphs reports that GraphRAG and ontology-guided KGs that integrate text chunks each achieved 90% answer accuracy (18/20), while a vector-RAG baseline reached 60% (12/20). The authors further note that ontology-guided KGs built from a relational database ontology perform competitively to text-derived ontologies, with practical advantages: the ontology learning step can be done once, reducing LLM usage costs, and it avoids the ontology-merging complexity that arises in purely text-based ontology extraction. This supports our decision to build the vocabulary layer first as a deterministic anchor structure and attach extracted rules and conditions to stable nodes afterward (da Cruz et al., 2025).

The approach cleanly separates two phases. Vocabulary building, the dictionary import, is deterministic, structured, and requires no LLM involvement. It creates the noun inventory with synonyms, translations, and taxonomic relationships. Knowledge attachment, the subsequent LLM extraction pipeline, processes source documents and attaches disposal rules, conditions, and facility links to existing anchor nodes. The LLM focuses on what it does well (understanding unstructured text) without the burden of entity resolution.

The Two-Label Strategy

The implementation of the two-label strategy relies on Neo4j's multi-label capability: a single node can carry both :DictionaryEntry and :WasteItem labels simultaneously, queryable via either. Dictionary entries are imported exclusively as :DictionaryEntry nodes. Promotion to :WasteItem happens only after LLM classification confirms that an entry represents a physical, disposable object, a separate phase that can be re-run as classification models improve or criteria change.

The practical consequence is that promotion is cheap and reversible. Adding a label is a single Cypher SET operation; removing one is equally trivial. This means the boundary between "recognized vocabulary" and "actionable waste item" can be adjusted without re-importing any data, and without affecting the synonym, translation, or taxonomy edges attached to the underlying DictionaryEntry node.

Data Sources

We selected three primary sources, prioritizing open licensing, structured bulk access, and complementary coverage.

Kaikki.org / Wiktextract provides the raw noun inventory. The project extracts structured data from Wiktionary using the wiktextract library and publishes it as JSONL files. We used the German Wiktionary extraction (de-extract.jsonl.gz, approximately 280 MB compressed), which contains 1.3 million entries across all languages and parts of speech defined in the German Wiktionary. Filtering for German-language nouns (lang_code == "de" and pos == "noun") yielded 355,660 entries. Each entry carries the headword, definitions (glosses), same-language synonyms, cross-language translations, hypernyms, hyponyms, and semantic categories. The data is licensed under CC-BY-SA 3.0 and GFDL.

OdeNet (Open German WordNet) provides taxonomic structure. Developed at the Hochschule Darmstadt, OdeNet organizes approximately 36,000 synsets (of which 25,767 are noun synsets) into hypernym-hyponym trees: "Schokoriegel" IS_A "Schokolade" IS_A "Suesswaren" IS_A "Lebensmittel." The wn Python package provides a unified query interface for both OdeNet and the English WordNet, and critically, OdeNet includes interlingual links (ILI) to the Open English WordNet that enable cross-language translation at the synset level. OdeNet was originally seeded from OpenThesaurus synonym clusters and Princeton WordNet, providing a bridge between colloquial German synonyms and formal taxonomic structure. Licensed under CC-BY-SA 4.0.

Open English WordNet (2024 edition) provides the English-side taxonomy, with approximately 120,000 synsets accessible through the same wn API. English DictionaryEntry nodes linked via TRANSLATION_OF edges to their German counterparts enable the chatbot agent to handle English queries natively through graph traversal rather than runtime translation.

Source selection was driven by practical constraints. We chose Kaikki.org over Duden or PONS because of its open license and bulk JSONL download; Duden is copyrighted and scraping would have required more effort whilst violating terms of service. We chose OdeNet over GermaNet because GermaNet, while considered the gold standard, requires a license application signed by a thesis advisor at Universität Tübingen, a bureaucratic dependency we could not afford given the short timeline of our project. OdeNet provides the same wn API, the same taxonomic structure, and a permissive CC-BY-SA license.

The Import Pipeline

The pipeline proceeds in four phases, each independently verifiable and re-runnable.

Phase 1: Download and Cache

The Kaikki.org JSONL file is downloaded via HTTP streaming and cached locally. OdeNet and the Open English WordNet are installed through the wn package, which manages its own SQLite database. All downloads are cached in data/dictionaries/ and reprocessed as needed. The kaikki-json Python package was evaluated but its built-in download URL returned a 404; we replaced it with direct requests streaming.

Phase 2: Parse and Filter

The raw JSONL (1.3 million entries) is filtered to German nouns (355,660 entries). For each entry, the pipeline extracts the headword, glosses, synonyms, translations, hypernyms, and semantic categories. These are cross-referenced against OdeNet synsets using synset IDs as deduplication keys.

The central processing step is concept clustering, which groups multiple surface forms into a single canonical concept. This proceeds in two passes. First, synset-based grouping: entries that share an OdeNet synset ID are merged into a single cluster. This produced 16,001 clusters from 28,816 matched words, entries like "Schokoriegel" and "Tafelschokolade" that OdeNet already groups under the same synset. Second, union-find synonym clustering: for the remaining entries (those without OdeNet synset matches), a union-find data structure groups words that share synonyms transitively. If word A lists word B as a synonym, and word B lists word C, all three are merged into one cluster. This second pass produced 313,230 additional clusters from the remaining entries.

For each cluster, a canonical term is selected using a heuristic: prefer the term that appears as a hypernym of the others in OdeNet; failing that, prefer the shortest single-word lemma. This favors "Schokolade" over "Schokoriegel" and "Batterie" over "Knopfzelle." The total output is 329,231 concept clusters written to an intermediate JSONL file (177 MB), each containing the canonical term, its synonyms, English translations, hypernyms, hyponyms, semantic categories, and source provenance. The pipeline runs in approximately 29 seconds after the initial download.

Phase 3: Neo4j Import

Rather than routing through the LLM extraction pipeline's EntityLoader, which is designed for single-entity dispatch from document processing, we implemented a dedicated five-pass UNWIND batch importer. Pass 1 creates canonical DictionaryEntry nodes. Pass 2 creates synonym nodes and SYNONYM_OF edges. Pass 3 creates English translation nodes and TRANSLATION_OF edges. Pass 4 creates hypernym nodes and IS_A edges. Pass 5 creates the Source node and DERIVED_FROM provenance edges. All operations use MERGE for idempotent re-runs. The import completes in approximately 20 seconds for all 329,231 clusters, roughly 100 times faster than routing through the single-entity loader would have been.

Phase 4: LLM Classification and Promotion

The 329,231 concept clusters are classified by a small language model (Llama 3.1 8B). For each entry, the model determines whether the noun refers to a physical object (is_physical), whether someone could plausibly need to dispose of it (disposal_relevant), and assigns a broad category (object, substance, food, organism, structure, person, place, event, or nonphysical).

A key optimization reduced the number of LLM calls by approximately 70%. Of the 329,231 clusters, roughly 237,000 are inflected forms (genitives, plurals, and datives) whose dictionary glosses follow patterns like "Nominativ Plural des Substantivs Schokolade." These forms inherit their classification from the base form without requiring an LLM call. The classifier detects inflections via regex, classifies the approximately 92,000 base forms through the LLM in batches of 25, then propagates results to all associated inflected forms.

An initial implementation bug caused approximately 104,000 inflected forms to fail base-form resolution: their glosses referenced a base form (such as "0,5-Liter-Flasche") that existed only as a synonym in another cluster, not as a canonical name. The fix was straightforward (extend the lookup table to map synonym names, not just canonical names, back to their cluster) but it affected a third of all entries. Because the classifier we've built supports resuming (re-running skips already-classified entries), the fix required only an 11-minute incremental run rather than repeating the full 98-minute initial classification. The total cost for classifying all 329,231 clusters was approximately $0.30 via Groq.

After classification, promotion is a single Cypher query: MATCH (d:DictionaryEntry {is_physical: true}) WHERE NOT d:WasteItem SET d:WasteItem. This adds the :WasteItem label to all entries classified as physical and disposal-relevant, making them instantly visible to all existing queries and tools that operate on the WasteItem label.

Results

The dictionary import produced the following graph.

Type	Value

DictionaryEntry nodes (German)	403,213
DictionaryEntry nodes (English)	20,503
Total nodes	423,717
SYNONYM_OF edges	122,960
TRANSLATION_OF edges	24,738
IS_A (taxonomy) edges	115,489
DERIVED_FROM (provenance) edges	325,937
Total edges	589,124
Promoted to :WasteItem	67,912

The node count exceeds the cluster count (329,231) because synonym, translation, and hypernym passes create additional nodes for terms that do not appear as canonicals in any cluster but are referenced as synonyms, translations, or hypernyms by other entries. The DERIVED_FROM count (325,937) is slightly below the cluster count because some clusters share UIDs after deduplication.

The 67,912 promoted WasteItem nodes represent a 124-fold increase over the previous graph's 548 items, transforming a sparse vocabulary into a broad foundation that covers the vast majority of physical objects a German-speaking user might ask about. The classification breakdown shows the distribution: approximately 55,000 objects, 9,000 substances, 3,700 food items, and smaller counts for organisms, structures, and other physical categories.

The taxonomy structure provides inheritance-based reasoning that the previous flat list could not support. A disposal rule attached to "Batterie" automatically applies to all hyponyms ("Knopfzelle," "AA-Batterie," "Lithium-Ionen-Akku") through IS_A traversal. The 122,960 synonym edges ensure that the chatbot agent can resolve colloquial, regional, and informal terms to their canonical forms. The 24,738 cross-language translation edges enable English-language queries to reach German disposal knowledge through graph traversal rather than runtime translation.

The entire pipeline, from raw download through classification and promotion, should cost less than 1€ and can be run in under two hours (when used with Groq), most of which will be the LLM classification phase. The deterministic steps (download, parsing, Neo4j import) complete in under a minute combined. This makes the dictionary baseline reproducible from scratch at negligible cost, an important property for a pipeline that may need to be re-run as dictionary sources update or classification criteria are refined.

Integration with Existing ETL

The dictionary baseline was designed as Phase 1 of the new graph, the first import after schema initialization. This required adapting the existing structured ETL modules to work against a dictionary-seeded graph rather than an empty one.

The facility import (src/etl/facilities.py) required no changes, since Facility nodes are independent of the vocabulary layer. The waste item import (src/etl/waste_items.py), however, posed a subtle problem. The existing pipeline uses MERGE (w:WasteItem {name: ...}) to create or match waste items. Before the dictionary import, this worked correctly, since every MERGE created a new node. After the dictionary import, "Backofen" already exists in the graph as a :DictionaryEntry node, but without the :WasteItem label, so the MERGE does not find it and creates a duplicate.

The solution was a pre-promotion step: before the main import loop, all CSV item names that exist as :DictionaryEntry nodes (but not yet as :WasteItem) receive the :WasteItem label. Then the existing MERGE finds them via the WasteItem label index and fires the ON MATCH branch, which updates the existing node with disposal relationships rather than creating a duplicate. For items not present in the dictionary, the MERGE creates new standalone :WasteItem nodes as before. We also added DERIVED_FROM provenance edges linking every CSV-imported item to a Source node, bringing the waste item importer in line with the provenance tracking pattern established by the dictionary pipeline.

This approach preserved backward compatibility (the same import code handles both dictionary-seeded and empty graphs) while preventing the duplicate nodes that would have undermined the entire rationale for top-down construction.

Infrastructure

The ETL pipelines described in the preceding sections require two things to run: a Neo4j database to write into and a Python environment to execute from. Both of these, along with a JupyterLab instance for interactive development and the chatbot application itself, are deployed on the workstation (10.18.2.105) that serves as the project's combined data processing and application server. All services (Neo4j, JupyterLab, and the chatbot) are deployed as rootless Podman containers managed by systemd, sharing the same machine and the same graph database. This section describes how these systems are deployed and how to reproduce or extend the existing setup.

Why Python

The entire data processing stack is implemented in Python. This was a deliberate choice driven by ecosystem fit rather than language preference. Python provides the tightest integration with the tools the project depends on: LangChain and LangGraph for the LLM extraction pipeline, the neo4j driver for database access, PyArrow for reading the multi-gigabyte Open Food Facts

Parquet dump, Pydantic for schema-driven extraction, and the wn package for WordNet access in the dictionary baseline. Equally important, Python's dominance in data science means that JupyterLab, our primary environment for developing and testing extraction pipelines, treats Python as a first-class citizen. Most of our pipelines were initially prototyped using Jupyter notebooks before being formalized into python modules.

Neo4j Deployment

Neo4j 5 Community Edition runs as a rootless Podman container managed by systemd through Podman's Quadlet mechanism. Quadlet files are declarative container definitions, similar in spirit to Docker Compose services but translated directly into native systemd units. This means Neo4j starts on boot, restarts on failure, and integrates with standard Linux service management (systemctl, journalctl) without requiring a container orchestrator.

The deployment consists of four Quadlet files installed to ~/.config/containers/systemd/: a container unit (neo4j.container) defining the image, ports (7474 for the HTTP browser, 7687 for the Bolt protocol), environment variables, and the APOC plugin configuration. Three companion volume units are used for persistent storage of graph data, logs, and the APOC import staging directory. We have created a install script (deployment/scripts/install.sh) that handles the complete setup: it verifies prerequisites (Podman installed, user lingering enabled), copies the Quadlet files, reloads the systemd daemon, enables the service, and polls the HTTP endpoint until Neo4j responds.

The choice of rootless Podman over Docker was motivated by the server environment. The workstation runs Fedora, where Podman is the native container runtime and rootless execution is the default, requiring no daemon, no root privileges, and no Docker socket to secure. Quadlet was chosen over a plain podman-compose setup (which also exists in the repository under docker/podman-compose.yml for local development) because it provides proper service supervision: automatic restarts, boot-time startup, and log aggregation through journald.

JupyterLab

A JupyterLab instance runs on the same workstation, providing an interactive environment for pipeline development, data exploration, and one-off extraction tasks. This co-location was necessary for practical reasons: the Open Food Facts Parquet dump is 4.4 GB of highly compressed columnar data representing 4.29 million products, the JSONL variant adds another 2.6 GB, and several intermediate processing files (the Kaikki dictionary JSONL at 280 MB, classification result caches, OFF subset extracts) add to the data footprint. Running notebooks on a laptop and connecting to a remote Neo4j instance would have meant either transferring these files repeatedly or maintaining duplicate copies. With JupyterLab on the same machine as Neo4j, both the data files and the database are local, and iteration cycles (load a Parquet partition, transform it, write to Neo4j, inspect the result) take seconds rather than minutes.

Every pipeline in this project has a corresponding notebook that served as its development environment: notebooks/pipeline_example.ipynb for the LLM extraction chain, notebooks/avv_classification.ipynb for the AVV code hierarchy, and additional notebooks for the dictionary baseline and Open Food Facts imports. These notebooks remain the most practical way to understand what each pipeline does, since they show the intermediate outputs that the command-line interfaces hide.

Database Management

Three operations cover the database lifecycle: initialization, dump import, and reset.

Initialization is handled by src/scripts/init_db.py, which verifies the Neo4j connection, applies schema constraints and indexes from src/db/schema.cql, and reports statistics (node counts by label, total relationships). The schema file defines uniqueness constraints on all node types, ensuring that MERGE operations in the ETL pipelines behave correctly, and indexes on frequently queried properties. Schema application is idempotent: the script parses schema.cql line by line and silently skips any constraint or index that already exists, so it can be re-run safely after a dump import, after manual schema changes, or as a post-deployment smoke test.

Dump import allows transferring a complete graph state between environments. The deployment/scripts/import-dump.sh script automates the process: it stops the Neo4j service (required by Community Edition for offline loading), mounts the dump file into a temporary container, runs neo4j-admin database load, restarts the service, and verifies the result by querying the node count.

Reset clears the entire database and is available via python src/scripts/init_db.py --reset. It requires interactive confirmation before executing, and is primarily used during development when a clean graph is needed for testing an import pipeline from scratch.

The Chatbot Deployment

The chatbot application is a separate repository containing the Angular frontend, FastAPI backend, and the LangGraph-based agent described in the second chapter. It ships with three deployment options: a Docker Compose stack for local development (docker/docker-compose.yml), a production Compose variant that pulls pre-built images from the GitHub Container Registry (docker/docker-compose.prod.yml), and a Helm chart (helm/fessi/) for Kubernetes deployment with StatefulSets for both PostgreSQL and Neo4j, configurable ingress, and secrets management.

In practice, the chatbot is deployed on the same workstation as a Podman Quadlet, loosely based on the Docker Compose definition but adapted for rootless systemd management, following the same pattern as the Neo4j deployment. Because all services share one machine, the backend connects to Neo4j at bolt://localhost:7687 and both the ETL pipelines and the chatbot agent operate on the same graph without any network transfer or synchronization step.

The PostgreSQL database that the chatbot uses for session management and user data is a separate concern; it does not interact with the knowledge graph.

Reproducing the Setup

Because the current deployment co-locates all services on a single workstation, the graph built by the ETL pipelines is immediately available to the chatbot agent. For future deployments where the data processing environment and the chatbot run on separate machines, the dump import mechanism described in earlier provides the transfer path: export a dump from the source Neo4j, transfer the file, and run the import script on the target. The dump captures the complete graph state, so the target environment does not need access to any of the raw data files or pipeline code.

The Neo4j browser is accessible at http://10.18.2.105:7474 and the Bolt endpoint at bolt://10.18.2.105:7687. The ETL pipeline's .env file requires three variables for database access (NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD) and, for LLM-dependent pipelines, LLM_API_KEY and LLM_BASE_URL (described in the companion Data Mining documentation). An .env.example template documents all available variables. Dependencies are installed via pip install -r requirements.txt; no system-level packages are required beyond Python 3.12 and Podman.

Results

The semester began with 548 waste items, 38 facilities, and a chatbot that could not answer most queries. It ended with a knowledge graph of over 5 million nodes and ~6 million edges, populated by nine distinct import pipelines, served from a single workstation, and immediately available to the chatbot agent.

	Start of Semester	**End of Semester**
WasteItem nodes	548	67,912
Total nodes	~590	~5,327,000
Total edges	~1,100	~5,950,000
Product coverage	0	4,290,312 products, 555,227 packaging components
Legal classification	0	1,640 AVV codes, 499 classified
Provenance-tracked sources	0	9 import pipelines

These numbers are the combined output of every pipeline described in this document and its companion: the dictionary baseline, Open Food Facts, EU Ecolabel, Open Beauty Facts, AVV codes, the Abfall-ABC, and a materials taxonomy. The graph grew from a sparse collection of municipal waste entries to a multi-layered knowledge base spanning vocabulary, products, legal classification, and disposal infrastructure. All pipelines are reproducible from open-access data sources using either local, self hosted or cloud inference APIs; no paid data sources or dedicated compute infrastructure is required.

Data Engineering Perspective

The following subsections discuss the key engineering decisions and their outcomes: how the schema evolved under pressure from real data, why the dictionary-first strategy paid off, how product decomposition works end-to-end, and what infrastructure choices made this feasible on a single workstation.

Schema as a living design

The graph schema began as a three-layer ontology (core entities, logic and rules, infrastructure) and grew to five layers over the course of the semester. Each extension was motivated by a concrete problem encountered during implementation, not by upfront speculation. The vocabulary layer (Layer 0) was added when the dictionary baseline strategy required nodes that were not yet waste items, German nouns that might or might not represent disposable objects. The product decomposition layer (Layer 2) was added when Open Food Facts data revealed that a single product could decompose into multiple packaging components, each routed to a different waste stream. Both extensions preserved the core traversal pattern unchanged: identify the item, resolve its disposal rule, evaluate conditions, arrive at a waste stream, locate the nearest collection point. The fact that this pattern survived two major schema extensions without modification validated the original design principles: layer separation, LLM-friendliness, and core-versus-professional layering.

The dictionary baseline as foundation

The decision to build a vocabulary layer before importing any domain-specific data proved to be the most consequential architectural choice of the semester. The conventional approach of importing sources one by one and deduplicating after the fact would have produced a graph where entity resolution errors compounded with every additional import. By seeding the graph with 329,231 concept clusters first, we established a broad anchor layer that subsequent imports could merge against rather than duplicate. The two-label strategy kept the graph clean: 67,912 entries received the WasteItem label after LLM classification, while the remaining vocabulary nodes gave the agent a fourth response state ("this is a recognized term but not something you'd dispose of") that the previous system could not express at all. The synonym, translation, and taxonomy edges, over 260,000 relationships connecting colloquial terms to canonical forms, German entries to English equivalents, and specific items to broader categories, are what transform a flat vocabulary list into a navigable structure. The entire pipeline is reproducible from scratch at negligible cost, which matters because it means the

vocabulary foundation can be rebuilt as classification criteria improve rather than being a one-time artifact.

Product decomposition chain

The full traversal path from barcode to per-component disposal route (Product -> HAS_COMPONENT -> PackagingComponent -> MADE_OF -> Material ->DEFAULT_STREAM -> WasteStream) is operational in the graph. The Open Food Facts import populated this chain with 4.29 million products and 555,227 packaging components. The barcode scanner module (pyzbar for detection, multi-API resolver for product lookup) has been built and tested against real products, demonstrating the end-to-end flow from photographing a barcode to receiving per-component disposal guidance. The barcode scanner is fully integrated into the chatbot as an agent tool, completing the chain from camera capture to per-component disposal instructions in a single interaction.

Infrastructure

All services (Neo4j, JupyterLab, the chatbot application, and the ETL pipelines) run on a single shared workstation managed through rootless Podman containers and systemd Quadlet units. This co-location means that the graph built by the ETL pipelines is immediately available to the chatbot agent without any synchronization or data transfer step. Every import pipeline is idempotent (MERGE-based, safe to re-run), provenance-tracked (DERIVED_FROM edges to Source nodes), and reproducible from documented data sources. The dump import mechanism provides the transfer path for future deployments where data processing and the chatbot run on separate machines.

Chatbot improvements

The visible reasoning panel, SSE streaming architecture, reasoning model switch, and expanded toolset described in Section 2 served a dual purpose. For end users, they transformed an opaque 30-second wait into a transparent, observable process. For the project itself, they made the knowledge graph evaluable, since a chatbot that cannot tolerate multi-step tool use cannot meaningfully traverse a knowledge graph. The web search fallback proved particularly valuable during development: it allowed us to validate the agent's reasoning architecture and identify which data categories users would need most, while the import pipelines were still being built. The agent's failed graph queries became the requirements list for the data sourcing strategy described in the companion Data Mining documentation.

Future Work

The pipeline infrastructure, graph schema, and data sourcing catalog produced this semester provide a foundation that subsequent project teams can build on without starting from scratch. The schema supports all anticipated data categories without modification. The ETL and LLM extraction pipelines are source-agnostic and extensible. What remains is primarily data work: running existing pipelines against assessed sources, importing data that requires institutional

access, and connecting the product layer to disposal knowledge. This section organizes the remaining work by effort and dependency.

Immediate Next Steps

The following tasks require no new infrastructure, no institutional partnerships, and no additional research. The pipelines exist and have been validated at proof-of-concept scale.

Completing the AVV classification

The hierarchical classification pipeline described in this document classified 500 WasteItem nodes against the AVV code hierarchy at a 99.8% success rate. The remaining ~67,000 WasteItem nodes can be processed with the same pipeline without modification. Classification quality will improve as the WasteItem population shifts from dictionary-promoted entries (which bias toward generic catch-all codes) to curated items from domain-specific sources.

Municipal vocabulary expansion

Berlin's BSR Open Data provides over 2,000 waste items with synonyms, avoidance tips, and recycling instructions under a CC BY license. Because the waste items themselves are universal across German cities, the BSR vocabulary can serve as a high-quality enrichment layer on top of the dictionary baseline, adding curated disposal knowledge to items that currently have only dictionary-level metadata. The import follows the same MERGE-based pattern as existing structured ETL pipelines.

Data Source Integration

The source list has over 400 sources, of which only a handful have been imported. The following categories represent what we think are the highest-priority remaining integrations.

Municipal SaaS APIs

The discovery that three platforms, Abfall+ (abfall.io), MyMüll (JumoMind), and Müllmax, consolidate most German municipal waste management is the single most actionable finding from the source research. Rather than building custom scrapers for each of Germany's 400+ districts, three API integrations would provide access to waste item vocabularies, sorting rules, collection schedules, and facility data for the majority of German university towns. The open-source Home Assistant integration (hacs_waste_collection_schedule) has already reverse-engineered these APIs and provides pre-configured tenant keys and municipality IDs for hundreds of cities, substantially reducing the integration effort. Imported municipal data would populate the disposal rules layer, currently the shallowest part of the graph, with city-specific sorting instructions and conditional logic.

Chemical substance databases

For a university with chemistry and biology laboratories, hazardous waste classification is a critical use case. GESTIS, the German occupational safety database maintained by the DGUV, provides 8,800 substance entries with storage classes, handling instructions, and regulatory references directly relevant to lab waste segregation. PubChem offers GHS hazard classifications for 111 million compounds, though the university context requires aggressive filtering to perhaps 500–2,000 common lab chemicals. The BAM dangerous goods database, which became freely accessible in July 2025, provides UN transport numbers and ADR classification codes for 4,500 substances. All three sources have been assessed and documented with access methods and format specifications.

LLM extraction from unstructured documents

The orchestrator pipeline, the four-stage LangGraph state machine for processing multi-entity documents, has been tested on individual documents but not yet run against the full corpus of sources identified during the research phase. Municipal waste guides, LAGA technical guidelines, university-specific chemical waste procedures, and AVV legal commentary documents remain as extraction targets. The pipeline infrastructure exists; the remaining work is prompt engineering for each document category and validation of extraction quality through dry-run iterations.

Institutional Partnerships

Two data gaps identified during the source evaluation cannot be closed through publicly available sources alone.

University procurement catalogs

The most significant data gap is the absence of an open database for everyday office and stationery items. German universities procure these items through framework contracts with B2B distributors who deliver product catalogs in the BMEcat XML format, a structured standard containing EAN barcodes, product descriptions, eCl@ss classification codes, manufacturer information, and links to Safety Data Sheets. These catalogs contain exactly the items the university purchases and needs to provide disposal guidance for. However, BMEcat catalogs are confidential B2B exchange documents, and access requires an institutional procurement account. A partnership with the university's procurement department is the most practical path to obtaining this data. A BMEcat parser would follow the same pattern as the existing structured ETL pipelines: parse XML, map fields to graph properties, MERGE into Neo4j with provenance tracking. The EAN-based deduplication already built into the product layer means that items appearing in both a BMEcat catalog and Open Food Facts would merge into a single enriched Product node rather than creating duplicates.

University-specific infrastructure data

Campus bin locations, per-room container assignments, building-specific waste rules, and Schadstoffmobil schedules are not published in any open dataset. Previous semesters attempted to collect this data through manual surveying and interviews with facilities management, with mixed results. The graph schema already supports this data through its infrastructure layer (Building → Room → Container nodes with WasteStream typing), and the disposal rule override mechanism (campus-scoped rules superseding city-level defaults) is designed for exactly this use case. The remaining challenge is establishing a reliable data collection process, ideally a collaboration with the university's facilities management department that provides periodic updates rather than a one-time manual survey.

Connecting Products to Disposal Knowledge

The product layer currently contains 4.29 million Product nodes and 555,227 PackagingComponent nodes with material links, but these products are not yet connected to the disposal knowledge in the core layer. The barcode scanner can decompose a product into its packaging components and route each component to a waste stream via the material taxonomy, but it cannot yet answer "How do I dispose of the product itself?" (as opposed to its packaging) without this connection.

Bridging this gap requires matching products to WasteItem categories: determining, for instance, that a jar of Nutella maps to the WasteItem "Schokoladencreme" (Biotonne) while its glass jar maps to "Altglas" (Altglascontainer) via the packaging decomposition chain. This matching can be approached in two ways: deterministic matching via Open Food Facts category tags (which map product categories like "Chocolates" to waste categories), or LLM-based matching where a language model infers the appropriate WasteItem from the product name and category. The deterministic approach is faster and cheaper; the LLM approach handles edge cases better. Both can run against the existing product and WasteItem nodes without schema changes.

Broader product database integration, incorporating UPCitemdb for non-food products and retail product data for electronics and household items, would extend the barcode scanner's coverage beyond food and cosmetics. The EAN-based deduplication mechanism ensures that products appearing in multiple sources merge correctly.

Graph Quality and Maintenance

As additional data sources are imported, two maintenance tasks will become necessary.

Material taxonomy extension

Of the 555,227 packaging components imported from Open Food Facts, 5,213 (1.1%) reference material tags are not present in the current 90-entry taxonomy. These were flagged during import rather than silently dropped. Extending the taxonomy to cover these tags and mapping

them to waste streams is a bounded task that improves the precision of the packaging decomposition chain.

Classification refinement

The AVV classification and dictionary baseline classification pipelines are both designed to be re-run as input data improves. The current classification results are biased by the dictionary-promoted WasteItem population, which contains many entries that are technically physical objects but not meaningful waste items. As curated waste items from the BSR vocabulary, municipal APIs, and domain-specific databases enter the graph, re-running the classification pipelines will produce more specific and useful results. The provenance tracking built into every pipeline ensures that re-classified entities can be traced back to their source, and the MERGE-based idempotency means re-runs are safe.

References

Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J. L., Vrgoč, D., & others. (2023). PG-Schema: Schemas for property graphs (arXiv:2211.10962v4). arXiv.

Curry, B. J. (2026, January 17). Building a knowledge graph: A comprehensive end-to-end guide using modern tools. Medium. https://medium.com/@brian-curry-research/building-a-knowledge-graph-a-comprehensive-end-to-end-guide-using-modern-tools-e06fe8f3b368

Chen, J., Wu, H., Pang, J., Wang, Y., Zhang, D., & Sun, C. (2025). Tool learning with language models: A comprehensive survey of methods, pipelines, and benchmarks. *Vicinagearth, 2*, Article 16. doi:10.1007/s44336-025-00024-x

da Cruz, T., Tavares, B., & Belo, F. (2025). *Ontology learning and knowledge graph construction: A comparison of approaches and their impact on RAG performance* (arXiv:2511.05991). arXiv. https://doi.org/10.48550/arXiv.2511.05991

Deep concept. (2026). *Top AI agentic workflow patterns that will lead in 2026: A practical look at the agentic patterns shaping real products not demos in 2026*. *Medium*. https://medium.com/@Deep-concept/top-ai-agentic-workflow-patten

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2025). *From local to global: A GraphRAG approach to query-focused summarization* [Preprint]. *arXiv*. arXiv:2404.16130v2.

Li, X., Zeng, W., Wang, Z., Zhu, D., Xu, J., Yu, W., & Zhou, J. (2024). GraphAr: An efficient storage scheme for graph data in data lakes (arXiv:2312.09577v4). arXiv.

Liu, C., Pavlenko, A., Interlandi, M., & Haynes, B. (2025). Data formats in analytical DBMSs: Performance trade-offs and future directions. The VLDB Journal, 34, Article 30. https://doi.org/10.1007/s00778-025-00911-1

Matsumoto, S., & Suda, Y. (2024). Accelerating TCP/IP communications in rootless containers using bypass4netns (arXiv preprint). arXiv.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision (arXiv:2212.04356). arXiv.

Raju, H. (2026, February 1). How to build a RAG system that actually works: Designing retrieval-augmented systems that hold up in the real world. Python in Plain English. https://python.plainenglish.io/how-to-build-a-rag-system-that-actually-works-04259eb3a66f

Saeedi, A., Peukert, E., & Rahm, E. (2020). Incremental multi-source entity resolution for knowledge graph completion. In The Semantic Web – ISWC 2020 (pp. 368–383). Springer. https://doi.org/10.1007/978-3-030-62419-4_21

Silfverskiöld, I. (2025, December 21). *How to build an over-engineered retrieval system: Which is actually how some people do it*. Data Science Collective. https://medium.com/data-science-collective/how-to-build-an-over-engineered-retrieval-system-923bd7466931

Singh, P. (2026). Rerankers in production: What actually moved the needle for my RAG systems. AI Mind.

Zeng, K., Li, C., Hou, L., Li, J., & Feng, L. (2021). A comprehensive survey of entity alignment for knowledge graphs. AI Open, 2, 1–13. https://doi.org/10.1016/j.aiopen.2021.02.002